

Swarm of Three

Agentic LLM Reasoning for Decentralised Autonomous Coordination

Area: Open-weight language models, chain-of-thought reasoning and peer-to-peer multi-agent coordination for small autonomous maritime teams.

The challenge

Most autonomy stacks rely on one central orchestrator commanding many subordinate platforms. It works until the link drops, until the mission has to be reshaped in language the orchestrator does not speak, or until the platforms have to argue among themselves about who does what. The single point of coordination is also the single point of failure.

Small open-weight reasoning models suggest another way: each asset carries its own reasoning substrate, accepts mission intent in natural language, and coordinates with its peers **by argument rather than by command** — no central planner, no privileged node. Whether that scales to a real autonomous maritime team, under changing intent and a partially observed world, is the open question this challenge puts to you. Build the smallest system that answers it honestly.

What you're building

A **decentralised, LLM-driven coordination layer** for three autonomous maritime assets in a shared simulated world. Each asset runs its own small open-weight language model. The **leader** receives the mission in natural language; from there the three agents reason, communicate and self-organise to execute it. **No central commander, no external planner.** Asset type (three USVs, three UAVs, a mixed trio) and world fidelity (2D, 3D, or ROS 2 + Gazebo) are the team's choice.

The system must demonstrate:

- **A natural-language mission interface** to the leader (a typed prompt is enough), propagated meaningfully to the peers — interpreted and re-expressed, not relayed verbatim.
- **Per-agent chain-of-thought** rendered visibly on screen, not buried in logs — the reasoning is the product.
- **Structured peer-to-peer messages** (proposals, acknowledgements, objections, hand-offs), also visible and inspectable.
- **A shared situational picture** — agents exchange what each one observes and reconcile disagreements (e.g. about a contact) into a common-enough view.
- **Adaptive behaviour** when the mission changes mid-execution, without restarting the system.
- **An honest representation of limits** — where the swarm breaks, you show it rather than hide it.

What you're judged on

Five tensions sit at the centre of decentralised agentic coordination. Your system is judged on whether it visibly tackles them, not on whether it generically “works”:

- **Reasoning legibility vs. latency** — chain-of-thought has to be visible and timely, yet three LLMs deliberating in real time is slow; show the reasoning without freezing the swarm.
- **Coherence without a commander** — three independent reasoners can deadlock, oscillate, or all grab the same task; with no orchestrator to break ties, the negotiation has to actually converge.
- **Grounding the language** — an LLM will fluently propose acting on a contact that does not exist; intent and plans must bind to the real world state, not to plausible text.
- **Adapting to changed intent** — a mid-mission change has to propagate across peers and the team has to re-converge, without a restart or quietly ignoring it.
- **Failure honesty** — when a peer goes silent or a model is uncertain, the swarm must surface it and route around it, not paper over the gap with confident-sounding text.

Reference mission scenarios

Four indicative reference missions. On the day, the jury runs the one you target and tests it with a mid-mission change of intent that arrives while the mission is already running. You need to handle one end-to-end (must-have); a second is a strong bonus.

Scenario	Mission, change of intent, and good handling
A — Patrol & anomaly report	“Patrol the marked area and report any vessel that does not match a commercial AIS pattern.” Change of intent: the jury extends the mission to a second area, or revises the reporting criterion. Good handling: divide the patrol, flag a non-conforming contact with a rationale, then re-divide coverage when the new instruction arrives.
B — Search with priority	“Find the missing buoy in this region, prioritise speed.” Change of intent: the priority flips from speed to full-coverage certainty mid-search. Good handling: partition the search to minimise time, then re-plan the pattern to trade speed for coverage once the priority changes.
C — Escort & formation	“Escort the marked vessel for 30 minutes, maintain a 500 m loose formation.” Change of intent: the jury re-tasks the team — a new vessel to escort, or a changed formation. Good handling: hold the loose formation, then re-form around the new instruction through visible coordination rather than each agent reacting alone.
D — Sequential investigation & rendezvous	“Investigate three points of interest in this order, then converge on the rendezvous point.” Change of intent: the jury reorders the points, or moves the rendezvous, mid-mission. Good handling: sequence and share the POIs and coordinate timing toward the rendezvous, then re-sequence cleanly when the order changes.

State the agents reason on

Whatever world you build, a credible swarm reasons over three families of state — pick the subset your chosen scenario needs:

- **Per-agent:** position, heading and speed; current task and its status; what the agent can sense now and its confidence in its own plan; its message inbox and outbox.
- **Mission (shared):** the mission intent and how it has changed; the agreed task-to-agent allocation; formation, rendezvous and timing constraints; safety geofences and de-confliction.
- **World:** contacts (synthetic or real AIS) with position and a behavioural label — for prioritisation, never targeting; area and point-of-interest geometry; environmental and traffic constraints; which peers can currently hear each other.

Stack & tools (suggestions, not requirements)

Open-weight LLMs (Llama 3.2 3B, Qwen 2.5 7B, Mistral 7B, Gemma 2) served locally via Ollama or LM Studio, or through free-tier APIs; an **agent loop** of your choice (LangChain, AutoGen, smolagents, or hand-rolled); a **world simulator** (Pygame, Three.js / WebGL, or ROS 2 + Gazebo); and an **inspectable peer-to-peer bus** (e.g. JSON over WebSocket or ROS 2 topics). Pick a model small enough to run three instances responsively on the clock. A fully synthetic world is fine — coordination, not map realism, is what scores.

Success criteria

Must-have

- Three agents, each driven by its own LLM instance.
- A natural-language mission interface, accepted by the leader and propagated meaningfully to the peers.
- Per-agent reasoning and peer-to-peer messages visibly rendered during execution.
- Demonstrable self-organisation on at least one reference scenario.
- Graceful adaptation to a mid-mission change of intent.
- A written account, in the README, of where the swarm breaks.

Should-have

- Multiple reference scenarios handled without code changes.
- Resilience to one agent failing or going silent mid-mission, shown live.
- Geographic plausibility — assets that respect terrain, traffic, weather or other realistic constraints.
- A “spectator view” mode that makes the reasoning legible to a non-technical observer.

Constraints and out of scope

Constraints

- Runs on a single laptop, with open-weight models served locally or free-tier APIs.
- Three agents, no central commander and no external planner — coordination must emerge peer-to-peer.
- Reasoning and messages must be visible and inspectable; a black-box swarm does not score.
- The human provides mission intent and may re-task, but does not micro-command individual agents.

Out of scope

- Not an operator-facing decision-support tool, and not a fleet-scale orchestrator — three is the deliberate working set.

- Not a study on LLM evaluation benchmarks — the artefact is a working coordinated swarm.
- Not a vehicle-control or flight-dynamics problem; no lethal targeting or autonomous engagement logic of any kind.

The demo moment we want to see

The 90-second jury demo. The participant types a mission to the leader in plain English. Three agents move across the world; around each, a thought bubble shows the live chain of reasoning; between them, structured messages flow — “I’ll take the western sector,” “I’ve lost the contact, can you swing south?,” “Confirmed, abandoning my pattern.” Mid-demo, the jury changes the mission, and the swarm reorganises before the audience finishes reading the new instruction. The judges should lean forward when they realise there is no commander in the loop — three independent reasoners took a sentence and turned it into coordinated behaviour, and you can follow every step.

Mentor and how to start

Each team is paired with a mentor combining naval operational experience with a software domain, there to validate scoping and plausibility — not to write code. With no starter kit, spend the first hour on setup: one open-weight model serving locally with three concurrent instances, three dots moving on a map, and a small JSON message schema (proposal / ack / objection / hand-off). Then wire one agent end-to-end and build the reasoning view alongside the sim — it is half the score.

Common pitfalls to avoid

- **Building a benchmark instead of a swarm** — the artefact is three agents coordinating live, not an evaluation harness.
- **Hiding the reasoning in logs** — chain-of-thought and messages on screen are half the score; if the jury has to read a terminal, you have lost the room.
- **Acting on fluent nonsense** — ground the language in world state; an agent chasing a contact that does not exist fails fast.
- **Sneaking in a central planner** — a “coordinator” that quietly assigns tasks defeats the point; coordination must be visible peer-to-peer.
- **Ignoring latency until the demo** — three LLMs in series will freeze a real-time world; design for responsiveness from hour one.
- **Waiting for data** — there is no dataset and no kit; stand up the world and one agent loop in hour one and iterate.